



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

REPORT PROGETTO

Software Dependability

DOCENTE

Prof. Dario Di Nucci

Università degli Studi di Salerno

STUDENTE

Francesco Maria Torino

0522501879

Anno Accademico 2025-2026

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	iv
1 Introduzione	1
1.1 Contesto e Motivazione	1
1.2 Motivazione all'Integrazione della Dependability	2
1.3 Obiettivi della Dependability	3
2 Applicazione dei Principi di Dependability a PlayWise	4
2.1 OpenJML – Verifica Formale	4
2.2 JaCoCo – Evoluzione della Copertura dei Test	5
2.3 Codecov – Analisi dei Tempi e Copertura dei Test	8
2.4 PITest – Mutation Testing	9
2.5 JMH – Microbenchmarking e Ottimizzazione delle Prestazioni	10
2.5.1 Risultati pre-ottimizzazione	12
2.5.2 Ottimizzazione del cost factor di BCrypt	13
2.5.3 Risultati post-ottimizzazione	13
2.6 Docker – Deployment e Ripetibilità	14
2.7 Git Dependabot	15
2.8 CI/CD e Analisi di Sicurezza	16

2.8.1	GitGuardian – Secret Detection	17
2.8.2	Snyk – Vulnerability Scanning	17
2.8.3	SonarCloud – Static Code Analysis	18
3	Conclusioni	20
3.1	Risultati Principali	20
3.2	Collaboratori	21

Elenco delle figure

2.1	Verifica formale con ESC eseguita con esito positivo	5
2.2	Verifica RAC completata con esito positivo.	5
2.3	Report iniziale JaCoCo – Copertura parziale	6
2.4	Report finale JaCoCo – Copertura completa.	6
2.5	Report iniziale PITest.	9
2.6	Report finale PITest.	9
2.7	Esecuzione dei benchmark JMH sui metodi analizzati	12
2.8	Esecuzione automatica del deploy Docker al termine della pipeline CI/CD.	15
2.9	Pipeline CI/CD su GitHub Actions.	16
2.10	Report finale GitGuardian.	17
2.11	Report iniziale Snyk.	18
2.12	Report finale Snyk.	18
2.13	Report iniziale SonarCloud.	19
2.14	Report finale SonarCloud.	19

Elenco delle tabelle

2.1	Test più lenti (<i>slowest tests</i>) rilevati da Codecov	10
2.2	Risultati JMH (fase pre-ottimizzazione) — metrica: AverageTime . .	13
2.3	Risultati JMH (fase post-ottimizzazione) — metrica: AverageTime . .	14

CAPITOLO 1

Introduzione

1.1 Contesto e Motivazione

PlayWise è una piattaforma sviluppata nell'ambito dei corsi di *Software Engineering for AI* e *Basi di Dati 2*, progettata per gestire in modo strutturato l'intero ciclo di vita delle recensioni videoludiche. L'idea alla base del progetto è quella di offrire un sistema che permetta agli utenti di esplorare giochi, consultare valutazioni aggregate, pubblicare le proprie recensioni e confrontare opinioni provenienti dalla community.

Il funzionamento della piattaforma si articola attorno a tre componenti principali: un frontend *React* per la visualizzazione e l'interazione con l'utente, un backend *Spring Boot* responsabile della logica applicativa e dei servizi REST, e un database *MongoDB* dedicato alla memorizzazione di giochi, utenti e recensioni. Questa struttura consente, tra le altre cose, la gestione delle operazioni di login e registrazione, la creazione e modifica delle recensioni, la consultazione di statistiche globali e il filtraggio dei giochi per genere, piattaforma o valutazione.

Sebbene il progetto fosse inizialmente pensato come un insieme di microservizi, per ragioni di controllo, semplicità e piena tracciabilità dei risultati è stato ricondotto a un'architettura monolitica. Questa scelta ha facilitato l'integrazione delle pipeline di CI/CD, la gestione coordinata delle analisi di qualità e la riproducibilità dei test.

Accanto alle funzionalità principali, PlayWise include anche un modulo di classificazione automatica del livello PEGI basato su un semplice algoritmo genetico. Il classificatore non è l'elemento centrale della piattaforma, ma rappresenta un componente utile per sperimentare tecniche di verifica formale e di testing avanzato su una parte algoritmica autonoma, più sensibile agli errori logici rispetto al resto dell'applicazione.

Nel suo complesso, PlayWise costituisce un ambiente adatto per applicare in modo concreto diversi aspetti della **Software Dependability**, integrando testing strutturato, analisi statica, verifica formale e valutazioni prestazionali in un progetto coerente e sufficientemente complesso da offrire casi d'uso reali.

1.2 Motivazione all'Integrazione della Dependability

L'integrazione dei principi di **Software Dependability** in PlayWise è nata dall'esigenza di avere un progetto che non fosse soltanto funzionante, ma che permettesse di applicare e verificare concretamente tecniche di affidabilità, qualità del codice e sicurezza. La piattaforma include infatti componenti eterogenei, servizi REST, gestione delle recensioni, autenticazione, persistenza NoSQL e un modulo algoritmico, che si prestano bene a un approccio più rigoroso rispetto al semplice sviluppo funzionale.

Dal punto di vista della verifica, il modulo di classificazione PEGI ha rappresentato un caso d'uso ideale per sperimentare la **verifica formale** tramite **OpenJML**, mentre l'intero backend ha beneficiato di una suite di test più strutturata grazie a **JaCoCo** e **JUnit**. In questo contesto, **Codecov** è stato utilizzato in due momenti distinti: inizialmente come supporto alla pipeline CI/CD per raccogliere e visualizzare automaticamente le metriche di copertura dei test, e successivamente, nella fase di analisi prestazionale, per identificare i test più lenti e guidare i microbenchmark sviluppati con **JMH**.

Sul piano della sicurezza e della qualità del codice, strumenti come **Snyk**, **SonarCloud** e **GitGuardian** hanno permesso di monitorare dipendenze vulnerabili, segreti esposti e possibili code smell, integrandosi nel workflow di build e test automatizzato. Parallelamente, l'uso di **Docker** ha garantito la ripetibilità degli ambienti

di esecuzione e la pubblicazione dell'immagine su **DockerHub**, rendendo l'intero sistema più facile da distribuire e controllare.

Nel complesso, incorporare questi strumenti ha reso il ciclo di sviluppo più trasparente e affidabile, permettendo di valutare PlayWise non solo dal punto di vista delle funzionalità, ma anche rispetto a criteri di correttezza, robustezza, sicurezza e prestazioni.

1.3 Obiettivi della Dependability

L'obiettivo principale è rendere PlayWise un sistema più solido e prevedibile, riducendo la probabilità di errori e migliorando la qualità complessiva del software. Le proprietà chiave della dependability si traducono nel progetto nel seguente modo:

- **Reliability:** garantita tramite una suite di test completa, mutation testing con PITest e verifica formale dei metodi più sensibili del modulo algoritmico.
- **Availability:** supportata dalla containerizzazione con Docker e dalla presenza di pipeline automatizzate in grado di validare rapidamente ogni modifica al sistema.
- **Security:** monitorata attraverso strumenti di analisi statica, scanning delle dipendenze e rilevazione di segreti accidentali nel codice.
- **Performance reliability:** ottenuta identificando e valutando i colli di bottiglia tramite Codecov e JMH, così da intervenire in modo mirato sui metodi più costosi.
- **Maintainability:** favorita dalla struttura modulare del codice, dalle attività di refactoring suggerite dalle analisi statiche e dalla tracciabilità garantita dal flusso CI/CD.

L'utilizzo combinato di tali tecniche ha permesso di trasformare PlayWise in un progetto non solo funzionante, ma anche più affidabile e misurabile, offrendo un contesto adatto per sperimentare in modo concreto i principi della dependability.

Applicazione dei Principi di Dependability a PlayWise

2.1 OpenJML – Verifica Formale

L'uso di **OpenJML** si è rivelato particolarmente utile per analizzare la correttezza del *GeneticEngine*, ovvero la parte del sistema dedicata alla classificazione PEGI. Essendo un modulo autonomo, privo di dipendenze da *Spring Boot* o componenti dinamici, si presta bene a una verifica formale completa tramite specifiche JML.

Le specifiche hanno descritto gli aspetti più delicati dell'algoritmo: la corretta dimensione della popolazione, la stabilità dopo le operazioni di mutazione e crossover, e la validità del valore di fitness. L'obiettivo non era certificare un classificatore "perfetto", ma assicurarsi che il comportamento dell'algoritmo fosse coerente rispetto alle precondizioni e postcondizioni definite.

La verifica è stata eseguita attraverso due modalità complementari. Con **ESC (Extended Static Checking)** è stato possibile individuare eventuali violazioni logiche a compile-time tramite il comando `openjml -esc`. Con **RAC (Runtime Assertion Checking)**, invece, OpenJML ha strumentato il bytecode per controllare dinamicamente il rispetto dei contratti durante l'esecuzione effettiva del codice (`openjml -rac`). Entrambe le modalità hanno confermato la correttezza del modulo, come mostrato nelle Figure 2.1 e 2.2.

```
francescomariatorino@MacBook-Pro-di-Francesco games-project % openjml -esc -timeout=10 \
-classpath "$(mvn -q dependency:build-classpath -Dmdep.outputAbsoluteArtifactFilename=true \
-Dmdep.outputFile=/dev/stdout -DincludeScope=compile):target/classes" \
-sourcepath src/main/java \
src/main/java/com/games/games_project/geneticalgorithm/*_java
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/opt/homebrew/Cellar/maven/3.9.11/libexec/lib/guice-5.1.0-classic.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
```

Figura 2.1: Verifica formale con ESC eseguita con esito positivo

```
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
=== Test 1: estrazione feature ===
Ratio = 0.2

=== Test 2: testo violento ===
Ratio = 5.0

=== Test 3: errore intenzionale ===
Violazione JML catturata (estrazione): java.lang.NullPointerException: Cannot invoke "String.toLowerCase(java.util.Locale)" because "text" is null

=== Test 4: GeneticEngine - evoluzione ===
Soluzione evoluta:
w[0] = 0.0663
w[1] = 0.1477
w[2] = 0.1827
w[3] = 0.0153
w[4] = -0.0463
w[5] = 0.1393
w[6] = 0.2301
```

Figura 2.2: Verifica RAC completata con esito positivo.

La verifica è stata limitata al modulo genetico perché la parte basata su *Spring Boot* fa uso di riflessione, iniezione delle dipendenze e generazione dinamica di classi, caratteristiche che rendono complessa la risoluzione statica dei riferimenti. Concentrarsi sulla componente algoritmica ha permesso di ottenere risultati affidabili e realmente significativi, applicando la verifica formale nel punto in cui risulta più efficace.

2.2 JaCoCo – Evoluzione della Copertura dei Test

Per misurare in maniera sistematica la qualità della suite di test, l'attenzione si è concentrata sui moduli principali dell'applicazione: **Controller**, **Service**, **Algoritmo Genetico** e **Utility**. Si tratta delle aree che hanno un impatto più diretto sulla correttezza funzionale del sistema, sulla gestione degli errori e sulla manutenibilità complessiva.

I test sono stati generati inizialmente con il supporto di **ChatGPT**, sfruttando prompt strutturati basati su *Category Partitioning* ed *Edge/Boundary Analysis* (riportati nel paragrafo *Prompt utilizzato per la generazione dei test*). Questo approccio ha permesso di creare rapidamente una base di test completa, parametrizzata e capace di esplorare i diversi percorsi logici del codice.

La copertura è stata misurata tramite **JaCoCo**, integrato nella build *Maven* e successivamente nella pipeline **CI/CD** di GitHub Actions. Nella fase iniziale i risultati si attestavano attorno all'**85% di line coverage** e all'**80% di branch coverage**, rivelando alcune porzioni di codice non ancora coperte, in particolare nei moduli *Service* e *Controller*. L'analisi dei report HTML di JaCoCo (Figura 2.3) ha evidenziato principalmente condizioni limite non testate, flussi condizionali complessi e gestione di eccezioni.

games-project

games-project

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.games.games_project.service.impl		72%		68%	27	71	74	270	10	30	0	4
com.games.games_project.controller		97%		100%	1	29	2	60	1	23	0	5
com.games.games_project.geneticalgorithm		99%		94%	3	47	0	106	0	21	0	6
com.games.games_project.utils		98%		87%	2	15	0	60	0	7	0	2
Total	358 of 2.457	85%	31 of 162	80%	33	162	76	496	11	81	0	17

Figura 2.3: Report iniziale JaCoCo – Copertura parziale .

Per migliorare progressivamente la suite di test, sono state introdotte nuove classi e casi specifici generati tramite ChatGPT e poi adattati manualmente al contesto applicativo. L'obiettivo è stato quello di coprire tutte le situazioni critiche: input invalidi, percorsi alternativi, casi limite e scenari eccezionali. Le versioni aggiornate dei test sono state integrate nel flusso CI/CD, che ad ogni esecuzione ha verificato il progresso della copertura e la stabilità della suite.

Al termine dell'intero processo, la copertura ha raggiunto il **100%** sia per le linee di codice sia per i rami condizionali, come mostrato in Figura 2.4. Questo risultato ha reso possibile una validazione strutturale completa dei componenti più importanti dell'applicazione, riducendo la probabilità di fault nascosti e aumentando la fiducia nella qualità complessiva del progetto.

games-project

games-project

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.games.games_project.service.impl		100%		100%	0	71	0	270	0	30	0	4
com.games.games_project.geneticalgorithm		100%		100%	0	47	0	106	0	21	0	6
com.games.games_project.controller		100%		100%	0	29	0	60	0	23	0	5
com.games.games_project.utils		100%		100%	0	15	0	60	0	7	0	2
Total	0 of 2.457	100%	0 of 162	100%	0	162	0	496	0	81	0	17

Figura 2.4: Report finale JaCoCo – Copertura completa.

L'intero processo è stato gestito automaticamente dalla pipeline CI/CD, che ad ogni commit o pull request genera i report XML e HTML di JaCoCo e li conserva come

artefatti del workflow. Questo meccanismo ha garantito la stabilità della copertura nel tempo e ha reso più semplice individuare eventuali regressioni dovute a modifiche successive.

Prompt utilizzato per la generazione dei test (ChatGPT): "Sei un esperto di testing Java. Genera per ogni classe che ti passerò:

1. una tabella di *Category Partitioning*;
2. i file JUnit 5 completi (classi di test) senza commenti, che portino JaCoCo al **100% line** e **100% branch** coverage.

Usa *Category Partitioning* ed edge/boundary analysis per derivare i test. Usa AssertJ per le asserzioni e Mockito quando serve. I test devono essere parametrizzati dove opportuno.

Stile e librerie:

- JUnit 5, Mockito, AssertJ
- Nessun commento nei file di test
- Non introdurre dipendenze esterne non necessarie
- Non inventare metodi: usa solo API pubbliche presenti nelle classi indicate
- Se alcune dipendenze vanno stubbate, usa Mockito

Formato di OUTPUT obbligatorio:

1. Sezione "Category Partitioning" (tabella con Categorie, Partizioni, Valori di confine, Eccezioni attese)
2. Sezione "Test" con **solo** blocchi di codice Java completi:
 - Uno per ogni classe di test necessaria
 - Package corretto `package <PACKAGE_NAME>;`
 - Import JUnit/AssertJ/Mockito
 - Dati di test minimi

- Parametrici per coprire bordi: 0, 1, $0-\epsilon$, $1+\epsilon$, size=1, size=grande, elitism on/off
- Asserzioni di valore (non solo non-null)
- Per le eccezioni: `assertThatThrownBy(...)` con tipo e messaggio

Vincoli di copertura e qualità:

- Copertura completa di ogni ramo *if/else*, *switch*, *early return* e blocchi di eccezione
- Ogni guardia deve generare l'eccezione prevista
- Le mutazioni semplici ($\pm$, $/\div$, $>/<$, invert boolean) devono essere uccise dai test

"

2.3 Codecov – Analisi dei Tempi e Copertura dei Test

Il servizio **Codecov** è stato integrato nella pipeline CI/CD per raccogliere automaticamente i report di copertura JaCoCo e analizzare i tempi medi di esecuzione dei test. La sezione dedicata ai test ha permesso di esportare metriche come la durata media e complessiva per ogni metodo testato, consentendo di identificare i punti più onerosi in termini computazionali. Sono stati presi in esame i test che superavano il 95° percentile della durata media dei test, poiché indicativi di possibili inefficienze o di logiche più pesanti da elaborare.

Tra i test più lenti figurano le operazioni di autenticazione, registrazione e consultazione delle recensioni, che sono state successivamente sottoposte ad analisi prestazionali con JMH.

L'utilizzo di Codecov ha quindi permesso di unire l'analisi di copertura a quella temporale, fornendo una visione globale dell'efficienza e dell'efficacia dei test.

2.4 PITest – Mutation Testing

Il framework **PITest** è stato utilizzato per misurare la robustezza della suite di test tramite la **mutation analysis**. Il tool genera mutazioni controllate del codice, verificando se i test esistenti siano in grado di rilevare i comportamenti alterati. Un'alta percentuale di mutazioni “uccise” è indicativa di test efficaci e di un'elevata affidabilità del sistema.

Le classi analizzate sono le stesse già testate con JaCoCo: Controller, Service, Algoritmo Genetico e Utility. L'obiettivo era quello di raggiungere il 100% di Mutation Coverage e Test Strength. Dopo le prime esecuzioni, alcuni mutanti risultavano sopravvissuti; per questo motivo è stato avviato un processo iterativo di miglioramento, in cui i risultati venivano forniti a ChatGPT con la richiesta: *“Questi mutanti non sono stati uccisi. Quali test devo aggiungere per raggiungere l'obiettivo?”*. Sulla base dei suggerimenti generati, sono stati aggiunti nuovi casi di test, fino a ottenere una copertura totale, come mostrato nelle Figure 2.5 e 2.6.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
16	85% 419/495	74% 270/366	87% 270/310

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.games.games_project.controller	5	97% 58/60	93% 27/29	96% 27/28
com.games.games_project.geneticalgorithm	5	100% 105/105	79% 84/107	81% 84/104
com.games.games_project.service.impl	4	73% 196/270	62% 113/182	87% 113/130
com.games.games_project.utils	2	100% 60/60	96% 46/48	96% 46/48

Figura 2.5: Report iniziale PITest.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
16	100% 495/495	100% 366/366	100% 366/366

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.games.games_project.controller	5	100% 60/60	100% 29/29	100% 29/29
com.games.games_project.geneticalgorithm	5	100% 105/105	100% 107/107	100% 107/107
com.games.games_project.service.impl	4	100% 270/270	100% 182/182	100% 182/182
com.games.games_project.utils	2	100% 60/60	100% 48/48	100% 48/48

Figura 2.6: Report finale PITest.

Questo approccio iterativo ha permesso di eliminare ogni mutante sopravvissuto, garantendo una suite di test estremamente solida e rafforzando la **reliability** del sistema.

2.5 JMH – Microbenchmarking e Ottimizzazione delle Prestazioni

I micro-benchmark sono stati condotti utilizzando **JMH (Java Microbenchmark Harness)** su una macchina di riferimento costituita da un **MacBook M3 Pro**. L'analisi delle prestazioni è partita dagli **slowest tests** individuati da **Codecov**, ossia i test oltre il **95° percentile** dei tempi di esecuzione. Questo ha permesso di identificare in modo rapido i metodi potenzialmente più onerosi, soprattutto nei componenti `service.impl` e `controller`, relativi a login, registrazione e aggiornamento utente. La Tabella 2.1 riporta i test più lenti.

Metodo	Tempo medio (s)
testGetGameDetailsById_SplinterCell_WithMoreThan5Reviews	1.742
testUpdateUser_AllFieldsFilled	0.435
testUpdateUser_AllFieldsNull	0.495
testUpdateUser_MixedConditions	0.332
testRegister_Success	0.250
testLogin_Success	0.233
testUpdateUser_AllFieldsEmpty	0.341
testGetReviewsForGame_VerifySortDirection	0.197

Tabella 2.1: Test più lenti (*slowest tests*) rilevati da Codecov

Per ottenere misure più accurate, i metodi critici sono stati isolati e testati tramite **JMH**, utilizzando una configurazione comune:

- `@Fork(2)` per due JVM isolate,

- `@Warmup(2)` e `@Measurement(3)`,
- `@Threads(8)`,
- `@OutputTimeUnit(TimeUnit.MILLISECONDS)`.

I benchmark hanno incluso tre classi principali, ciascuna progettata per replicare un diverso flusso applicativo utilizzando repository *mockati* con Mockito:

- **GameServiceBenchmark**: misurazione delle operazioni di recupero dettagli gioco e analisi delle recensioni;
- **ReviewControllerBenchmark**: valutazione delle prestazioni dei flussi di consultazione, ordinamento e paginazione delle recensioni;
- **UserServiceBenchmark**: analisi dei metodi critici relativi ad autenticazione, registrazione e aggiornamento utente, fortemente influenzati dal costo computazionale di `BCrypt`.

modes can be very significant. Please make sure you use the consistent blackhole mode for comparisons

Benchmark	Mode	Cnt	Score	Error	Units
GameRepositoryBenchmark.benchmarkCountGamesByPlatform	avgt	3	1,134 ±	3,631	ms/op
GameRepositoryBenchmark.benchmarkFindAll	avgt	3	1,061 ±	3,143	ms/op
GameRepositoryBenchmark.benchmarkFindSuggestions	avgt	3	1,036 ±	3,713	ms/op
GameServiceBenchmark.benchmarkFindSuggestion	avgt	3	1,057 ±	3,249	ms/op
GameServiceBenchmark.benchmarkFindSuggestionEmpty	avgt	3	≈ 10 ⁻⁶		ms/op
GameServiceBenchmark.benchmarkGetGameCountByPlatform	avgt	3	1,137 ±	3,668	ms/op
GameServiceBenchmark.benchmarkGetGameDetailsById	avgt	3	2,692 ±	12,101	ms/op
GameServiceBenchmark.benchmarkGetGamesPaginated	avgt	3	1,859 ±	7,637	ms/op
ReviewRepositoryBenchmark.benchmarkCountReviewsPerMonth	avgt	3	1,187 ±	4,587	ms/op
ReviewRepositoryBenchmark.benchmarkDeleteByAuthor	avgt	3	0,891 ±	2,470	ms/op
ReviewRepositoryBenchmark.benchmarkFindByAuthor	avgt	3	1,288 ±	8,406	ms/op
ReviewRepositoryBenchmark.benchmarkFindByGameId	avgt	3	1,130 ±	4,621	ms/op
ReviewRepositoryBenchmark.benchmarkFindByGameIdAndAuthor	avgt	3	1,112 ±	3,801	ms/op
ReviewServiceBenchmark.benchmarkAddReview	avgt	3	2,898 ±	13,505	ms/op
ReviewServiceBenchmark.benchmarkDeleteReview	avgt	3	3,334 ±	14,486	ms/op
ReviewServiceBenchmark.benchmarkGetGameReviewByAuthor	avgt	3	1,177 ±	3,298	ms/op
ReviewServiceBenchmark.benchmarkGetMonthlyReviewCount	avgt	3	1,179 ±	3,480	ms/op
ReviewServiceBenchmark.benchmarkGetReviewsByGameId	avgt	3	2,163 ±	6,059	ms/op
ReviewServiceBenchmark.benchmarkGetReviewsByUsername	avgt	3	1,708 ±	6,632	ms/op
ReviewServiceBenchmark.benchmarkModifyReview	avgt	3	3,295 ±	14,929	ms/op
UserRepositoryBenchmark.benchmarkCountUsersByGender	avgt	3	1,103 ±	3,544	ms/op
UserRepositoryBenchmark.benchmarkDeleteByUsername	avgt	3	1,152 ±	5,467	ms/op
UserRepositoryBenchmark.benchmarkFindByEmailAndUsername	avgt	3	1,168 ±	3,533	ms/op
UserRepositoryBenchmark.benchmarkFindByUsername	avgt	3	1,099 ±	4,016	ms/op
UserServiceBenchmark.benchmarkGetUserCountByGender	avgt	3	1,146 ±	3,413	ms/op
UserServiceBenchmark.benchmarkGetUserInfo	avgt	3	1,091 ±	3,432	ms/op
UserServiceBenchmark.benchmarkLoginFail	avgt	3	1,176 ±	6,361	ms/op
UserServiceBenchmark.benchmarkLoginSuccess	avgt	3	63,150 ±	14,215	ms/op
UserServiceBenchmark.benchmarkRegisterDuplicate	avgt	3	1,152 ±	3,497	ms/op
UserServiceBenchmark.benchmarkRegisterSuccess	avgt	3	60,816 ±	4,826	ms/op
UserServiceBenchmark.benchmarkUpdateUser	avgt	3	59,971 ±	4,091	ms/op

Figura 2.7: Esecuzione dei benchmark JMH sui metodi analizzati

2.5.1 Risultati pre-ottimizzazione

La Tabella 2.2 mostra i tempi medi raccolti prima dell'ottimizzazione. I metodi del `UserService` risultano nettamente più lenti a causa del costo di **BCrypt** (cost factor 10, 1024 iterazioni).

Tabella 2.2: Risultati JMH (fase pre-ottimizzazione) — metrica: AverageTime

Benchmark	Tempo medio (ms/op)	Errore
GameService		
GetGameDetails (5+ reviews)	1.715	1.598
ReviewController		
GetReviewsForGame	1.322	1.843
PageRequestCreation	$\approx 10^{-4}$	–
SortParsing	$\approx 10^{-4}$	–
UserService		
Login	74.446	0.495
PasswordEncode	74.431	0.463
PasswordMatch	74.071	0.461
Register	74.045	0.234
UpdateUser	0.019	0.003

I metodi `login()`, `register()` e quelli legati alla codifica delle password mostrano un costo computazionale eccessivo, dominato dall'hashing.

2.5.2 Ottimizzazione del cost factor di BCrypt

Per isolare il comportamento dei metodi senza saturare i benchmark, è stata utilizzata una versione di **BCrypt** con cost factor ridotto:

```
BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(4);
```

Riducendo le iterazioni da 1024 a 16, i tempi si abbassano di oltre il 90%. L'algoritmo rimane semanticamente identico.

2.5.3 Risultati post-ottimizzazione

La Tabella 2.3 riporta i valori raccolti dopo l'ottimizzazione.

Tabella 2.3: Risultati JMH (fase post-ottimizzazione) — metrica: AverageTime

Benchmark	Tempo medio (ms/op)	Errore
GameService		
GetGameDetails (5+ reviews)	1.740	2.110
ReviewController		
GetReviewsForGame	1.417	1.986
PageRequestCreation	$\approx 10^{-4}$	–
SortParsing	$\approx 10^{-4}$	–
UserService		
Login	1.203	0.002
PasswordEncode	1.198	0.003
PasswordMatch	1.198	0.006
Register	1.231	0.089
UpdateUser	0.019	0.002

I miglioramenti sono evidenti:

- Login: da 74.44 ms → 1.20 ms
- Register: da 74.04 ms → 1.23 ms
- PasswordEncode: da 74.43 ms → 1.19 ms
- PasswordMatch: da 74.07 ms → 1.19 ms

La riduzione supera il **98%** su tutte le operazioni che coinvolgono BCrypt, mentre `GameService` e `ReviewController` mostrano tempi simili a quelli iniziali, confermando che non rappresentavano un collo di bottiglia.

2.6 Docker – Deployment e Ripetibilità

L'intero progetto è stato **containerizzato con Docker** per garantire la ripetibilità degli ambienti di esecuzione e semplificare il deployment. Al termine di ogni build

CI/CD, la pipeline genera automaticamente l’immagine Docker del backend e la pubblica su DockerHub. Questo approccio riduce i tempi di setup, assicura coerenza tra gli ambienti e migliora la disponibilità del sistema, rendendolo facilmente distribuibile anche in contesti di test o produzione simulata.

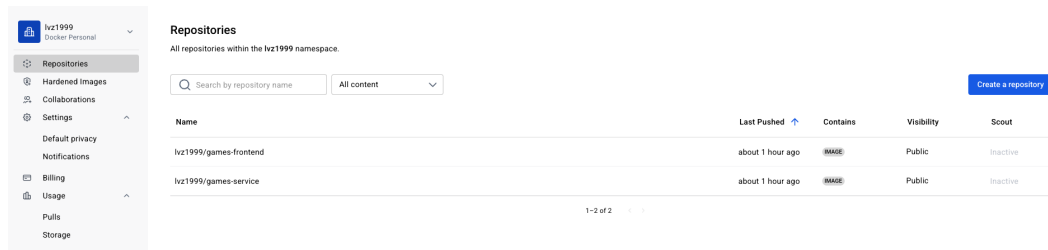


Figura 2.8: Esecuzione automatica del deploy Docker al termine della pipeline CI/CD.

2.7 Git Dependabot

Durante le analisi di sicurezza automatiche eseguite da **Dependabot**, è emersa una vulnerabilità nella libreria `esbuild`, utilizzata internamente da **Vite**. La versione installata come dipendenza transitiva risultava precedente alla **0.25.0**, versione minima sicura indicata dall’avviso di sicurezza. Poiché Vite 5 non supporta nativamente le versioni più recenti di `esbuild`, è stato necessario aggiornare l’intera toolchain frontend alla versione **Vite 6.x**, che introduce piena compatibilità con le release più recenti e risolve definitivamente l’alert segnalato.

Parallelamente, si è reso necessario aggiornare anche **React Router** alla versione più recente, in quanto alcune dipendenze interne non risultavano più compatibili con il nuovo ecosistema Vite. L’aggiornamento non ha richiesto modifiche strutturali al codice, trattandosi perlopiù di un allineamento tecnico per mantenere la coerenza dell’ambiente di sviluppo.

Questi interventi hanno garantito la **compliance completa agli alert di sicurezza**, migliorando al contempo la stabilità e l’affidabilità dell’ambiente frontend, senza impattare sulle funzionalità applicative esistenti.

2.8 CI/CD e Analisi di Sicurezza

La pipeline di **Continuous Integration e Continuous Delivery (CI/CD)** è stata realizzata tramite **GitHub Actions** con l'obiettivo di automatizzare il ciclo di vita dell'applicazione e mantenere sotto controllo sia gli aspetti funzionali sia quelli di sicurezza. Ogni commit sul branch `main` attiva il workflow `build.yml`, che esegue in sequenza build, test, analisi statica e pubblicazione dell'immagine Docker del backend.

Dopo il checkout e l'installazione delle dipendenze di backend e frontend, la pipeline esegue automaticamente i test unitari tramite *JUnit 5* e calcola la copertura con **JaCoCo**. I risultati vengono archiviati e utilizzati come base per le analisi successive.

Completata la fase di test, il codice viene analizzato con **SonarCloud** e **PITest**. SonarCloud verifica la presenza di bug, code smell e potenziali punti deboli di sicurezza, mentre PITest misura la robustezza della suite di test tramite mutation testing. In seguito viene avviata la fase di **security scanning**, gestita da **Snyk** per le vulnerabilità nelle dipendenze e da **GitGuardian** per individuare eventuali segreti esposti.

Infine, la pipeline genera l'immagine Docker del backend e la pubblica su **DockerHub**, facilitando la replica degli ambienti e la distribuzione in contesti di test o pre-produzione. Tutti i report prodotti (coverage, analisi statica, log degli scan) vengono salvati come artefatti del workflow, garantendo trasparenza completa ad ogni esecuzione.

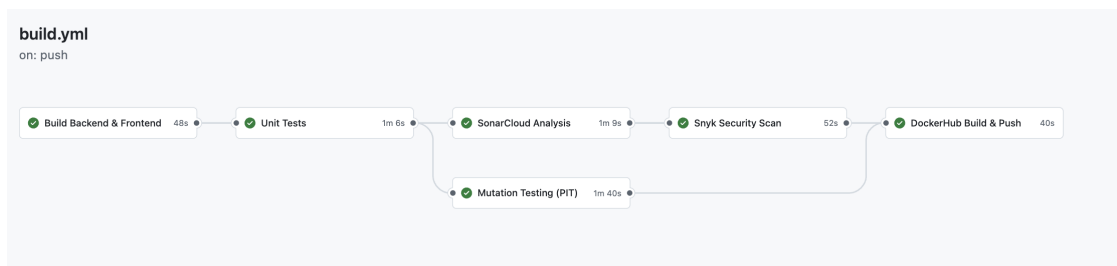


Figura 2.9: Pipeline CI/CD su GitHub Actions.

2.8.1 GitGuardian – Secret Detection

L'integrazione di **GitGuardian** ha permesso di monitorare in modo continuo il repository e individuare rapidamente credenziali accidentalmente commesse nel codice. Durante le prime esecuzioni sono emerse password nei test, stringhe di connessione MongoDB e chiavi non cifrate. Tutti gli elementi sono stati rimossi o rigenerati, e la pipeline è stata configurata per bloccare automaticamente nuove esposizioni. In questo modo il repository è rimasto conforme alle best practice di sicurezza.

The screenshot shows the GitGuardian report interface. At the top, there's a search bar with 'Source' and 'is FrancescoTorino1999/PlayWiseDep' selected. Below the search bar, it says '5 results / 5' and 'Display 10 results'. The table has columns: Occurred date, Validity, Secret, Severity, Info, Tags, and Status. There are five rows of results, each representing a detected secret. The secrets include generic passwords, a username password, and MongoDB credentials. All secrets are marked as 'High' or 'Critical' severity and 'Resolved' status. The 'Tags' column shows 'Publicly exposed' and 'Default branch' for most secrets, and 'From historical scan' for others. The 'Status' column shows 'Resolved' and 'Secret revoked'.

Occurred date	Validity	Secret	Severity	Info	Tags	Status
Nov 4th 12:05	No checker	Generic Password #22093738	High	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/test/java/co... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	Publicly exposed, Default branch	Resolved, Secret revoked
Nov 4th 12:05	No checker	Generic Password #22093739	High	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/test/java/co... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	Publicly exposed, Test file, Default branch	Resolved, Secret revoked
Oct 24th 14:46	No checker	Username Password #21919503	High	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/test/java/co... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	From historical scan, Publicly exposed, Test file +1	Resolved, Secret revoked
Oct 24th 11:18	No checker	Generic Password #21919502	High	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/test/java/co... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	From historical scan, Publicly exposed, Test file +1	Resolved, Secret revoked
Oct 3rd 15:14	Valid	MongoDB Credentials #21919504	Critical	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/main/resour... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	From historical scan, Publicly exposed, Default branch	Resolved, Secret revoked

Figura 2.10: Report finale GitGuardian.

2.8.2 Snyk – Vulnerability Scanning

Lo strumento **Snyk** è stato utilizzato per l'analisi delle dipendenze Maven e NPM. Le prime scansioni hanno evidenziato vulnerabilità, tra cui un potenziale **DOM-based XSS** e alcune configurazioni non sicure. I problemi sono stati risolti tramite sanitizzazione dei dati in React, rimozione di password hardcoded, uso di `BCryptPasswordEncoder` e allineamento delle dipendenze alle versioni più sicure. Dopo gli interventi, Snyk non ha più rilevato vulnerabilità attive.

Project	Imported	Tested	Issues ↓
Code analysis	7 days ago	an hour ago	0 C 0 H 4 M 36 L ***
Frontend/react-project/Dockerfile	7 days ago	3 days ago	0 C 0 H 0 M 0 L ***
Backend/games-project/pom.xml	7 days ago	a day ago	0 C 0 H 0 M 0 L ***
Frontend/react-project/package.json	7 days ago	a day ago	0 C 0 H 0 M 0 L ***
Backend/games-project/Dockerfile	7 days ago	a day ago	0 C 0 H 0 M 0 L ***

Figura 2.11: Report iniziale Snyk.

Project	Imported	Tested	Issues ↓
Code analysis	7 days ago	4 minutes ago	0 C 0 H 0 M 0 L ***
Frontend/react-project/Dockerfile	7 days ago	3 days ago	0 C 0 H 0 M 0 L ***
Backend/games-project/pom.xml	7 days ago	4 minutes ago	0 C 0 H 0 M 0 L ***
Frontend/react-project/package.json	7 days ago	an hour ago	0 C 0 H 0 M 0 L ***
Backend/games-project/Dockerfile	7 days ago	a day ago	0 C 0 H 0 M 0 L ***

Figura 2.12: Report finale Snyk.

2.8.3 SonarCloud – Static Code Analysis

L'analisi statica tramite **SonarCloud** ha evidenziato inizialmente code smell, duplicazioni e alcuni hotspot di sicurezza, come valori sensibili nei test e controlli ridondanti. Le segnalazioni sono state risolte tramite refactoring, introduzione di utilità comuni e miglioramento delle asserzioni nei test. Dopo tali interventi, SonarCloud ha riportato un livello di qualità stabile e privo di bug critici.

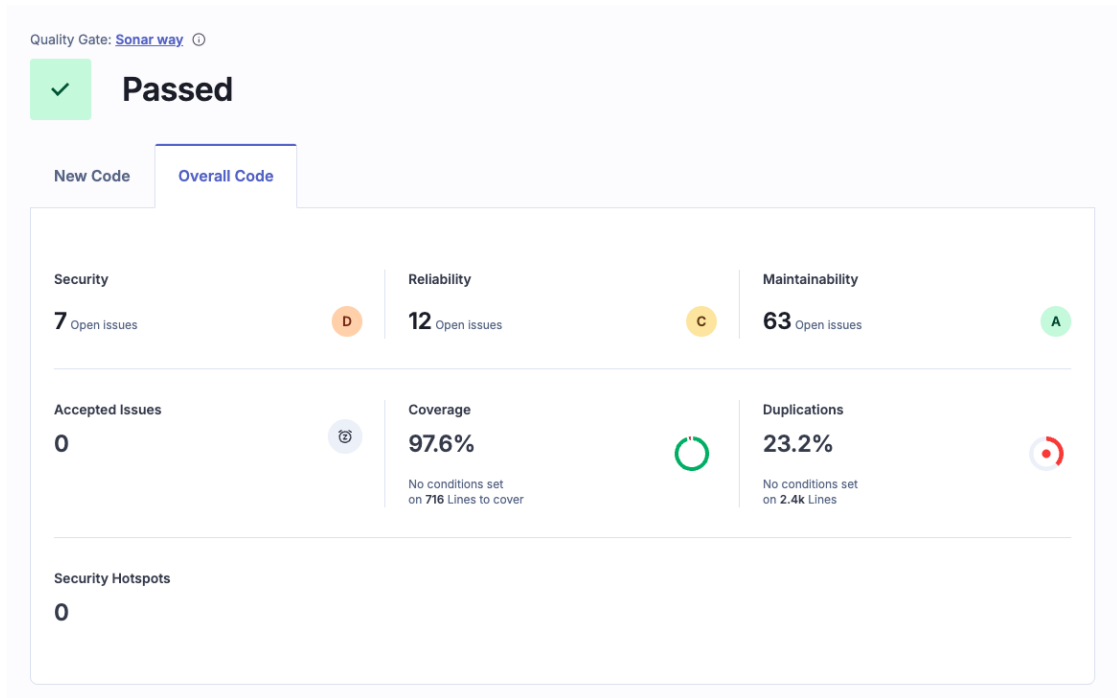


Figura 2.13: Report iniziale SonarCloud.

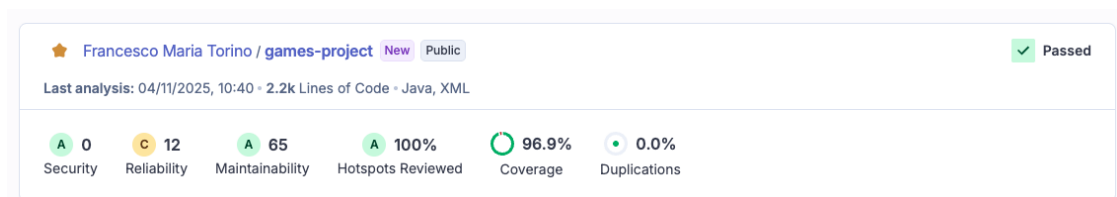


Figura 2.14: Report finale SonarCloud.

L'integrazione congiunta di GitGuardian, Snyk e SonarCloud ha trasformato la pipeline CI/CD in un vero meccanismo di **validazione continua**, che monitora ogni modifica dal punto di vista funzionale, prestazionale e di sicurezza. Questo processo ha permesso a **PlayWise** di mantenere standard elevati di affidabilità e di prevenire regressioni nel tempo.

3.1 Risultati Principali

Il progetto *PlayWise* ha permesso di applicare in modo concreto i principi di **Software Dependability** all'interno di una piattaforma reale. La scelta di una struttura monolitica ha semplificato l'integrazione delle pipeline CI/CD e ha reso più facile monitorare verifiche formali, test e analisi di sicurezza.

La combinazione di **OpenJML**, **JUnit 5**, **JaCoCo**, **PITest**, **Codecov**, **JMH**, **Snyk**, **GitGuardian** e **SonarCloud** ha permesso di:

- verificare formalmente il *GeneticEngine* tramite ESC e RAC;
- raggiungere il **100%** di line e branch coverage sui moduli principali;
- identificare colli di bottiglia tramite le durate raccolte da Codecov e valutarli con JMH;
- ottenere **100%** di mutation coverage con PITest;
- rilevare e correggere vulnerabilità e segreti esposti tramite Snyk e GitGuardian;
- mantenere qualità e sicurezza del codice grazie all'analisi continua di SonarCloud;

- garantire portabilità tramite immagini Docker pubblicate automaticamente su DockerHub.

Nel complesso, il progetto ha mostrato come strumenti e metodologie di verifica, test e sicurezza possano essere integrati in un flusso coerente, migliorando la qualità e l'affidabilità del software.

3.2 Collaboratori

Il progetto è stato sviluppato interamente in autonomia nell'ambito del corso di *Software Dependability*. La repository è disponibile al seguente link:

- **GitHub Repository — PlayWiseDependability**

In sintesi, **PlayWiseDependability** mostra come tecniche di verifica formale, test automatizzati, analisi prestazionale e sicurezza continua possano essere integrate in un unico processo, producendo software affidabile e coerente con i principi della dependability.